Document number	P3230R2
Date	2025-02-27
Audience	LEWG, SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

`views::unchecked_(take|drop)`

Abstract

This paper proposes two Tier 1 adaptors in P2760: `views::unchecked_drop` and `views::unchecked_take`, cousins of `drop` and `take`, to improve the C++26 ranges facilities, which were renamed to `views::unchecked_take` and `views::unchecked_drop` after Tokyo WG21, as "unchecked" better describes the utility's nature than "exactly".

Revision history

R2

- Added *Preconditions* to the wording based on SG9 feedback.
- Aligned proposed changes with the latest draft.

R1

- Added *unchecked* part for the naming.
- Added benchmarks.
- Remove `empty_view` specialization for return types.

R0

Initial revision.

Discussion

`unchecked_take` and `unchecked_drop` are very similar to `take` and `drop`. The only difference is that the former requires the selection size to be no larger than the size of the original range; otherwise, it is *Undefined Behavior*.

Since there is no boundary check, `views::unchecked_`*`meow`* is more efficient than `views::`*`meow`* for situations where the user already knows that there are enough elements in the range, which is what "unchecked" is all about.

Design

Should we name it `views::unchecked_`*`meow`* or `views::`*`meow`*`_unchecked`?

To emphasize the "unchecked" part, it is natural to think about whether to put it before or after "take".

The author prefers to `unchecked_`*`meow`* to align with the naming convention in the current standard, given that we already have `inplace_vector::unchecked_push_back`. If we put it at the end, it should be more accurate to name it *`meow`*`_uncheckly`, however,

"uncheckly" is not quite a common word.

Should we introduce `unchecked_meow_view` classes?

Although both can achieve similar effects with existing utilities, e.g. `unchecked_take(r, N)` is somewhat equivalent to `views::counted(ranges::begin(r), N)`, and `unchecked_drop` is somewhat equivalent to `subrange(ranges::next(ranges::begin(r), N), ranges::end(r))`, these are not fully replaceable due to certain limitations.

The main problem is that both require manually extracting iterators of the range, and such iterator-based approach causes dangling when applied to rvalue ranges.

Since we've already assumed that the range is large enough, there is no need to consider out-of-bounds cases in the implementation, which means that `unchecked_meow_view` is just a simplified version of `meow_view`, introducing them doesn't bring much complexity as the `meow_view` already not that complicated.

Should we specialized for return types?

Currently, both `take` and `drop` have optimized the return type. When they take an object of range type `empty_view`, `span`, `string_view`, `subrange`, `iota_view`, and `repeat_view`, it will return an object of the same type to reduce template instantiation. There is no reason not to apply similar optimizations to new ones. Noted that the author removed the specialization for `empty_view` as the value of `N` can only be `0`.

In addition, `views::unchecked_meow` applied to infinite ranges now can downgrade to finite ranges. For example, `views::iota(0) | views::unchecked_take(5)` will produce `views::iota(0, 5)`, and `views::iota(0) | views::unchecked_drop(5)` will produce `views::iota(5)`. This also applies to infinite `subrange`s, which can be considered as an improvement.

Implementation experience

The author implemented `views::unchecked_(take|drop)` based on libstdc++, see here.

Benchmarks

One of the advantages of `views::unchecked_take` over `views::take` is that it always produces a sized range even if the original range is a non-size range such as `generator` or `istream_view`, this can further benefit the container construction downstream as size information implies the reserve size.

In other words, `r | views::unchecked_take(N) | ranges::to<C>` would have better performance than `r | views::take(N) | ranges::to<C>`. The following table shows that for the input-only range, constructing a `vector` after applying `views::unchecked_take` is ~3.8 times faster than `views::take` in terms of Iterations (see here):

Table — Performance comparison of constructing `vector` via `views::take` vs. `views::unchecked_take`

Benchmark	Time	CPU	Iterations
`construct_from_take<std::vector<int>>`	17454461 ns	9032396 ns	59
`construct_from_unchecked_take<std::vector<int>>`	7671973 ns	3164699 ns	222

`views::unchecked_drop` also has better performance when applied to non-sized ranges, as the new `begin()` for all random-access ranges can be obtained in constant time by just calculating `r.begin() + N`. The following table shows that for non-sized null-terminated raw string, constructing a `string` after applying `views::unchecked_drop` is ~1.6 times faster than `views::drop` in terms of Iterations (see here):

Table — Performance comparison of constructing `string` via `views::unchecked_drop` vs. `views::drop`

Benchmark	Time	CPU	Iterations
`construct_from_drop<std::string>`	9762106 ns	5644617 ns	121
`construct_from_unchecked_drop<std::string>`	6812259 ns	4149667 ns	192

Proposed change

This wording is relative to latest working draft.

1. Add two new feature-test macros to 17.3.2 [version.syn]:

```
#define __cpp_lib_ranges_unchecked_take 20XXXXL // freestanding, also in <ranges>

#define __cpp_lib_ranges_unchecked_drop 20XXXXL // freestanding, also in <ranges>
```

2. Modify 25.2 [ranges.syn], Header `<ranges>` synopsis, as indicated:

```
#include <compare>              // see [compare.syn]
#include <initializer_list>     // see [initializer.list.syn]
#include <iterator>             // see [iterator.synopsis]

namespace std::ranges {
  […]
  namespace views { inline constexpr unspecified take = unspecified; }              // freestanding

  // [range.unchecked.take], unchecked_take_view
  template<view> class unchecked_take_view;                                         // freestanding

  template<class T>
    constexpr bool enable_borrowed_range<unchecked_take_view<T>> =                  // freestanding
      enable_borrowed_range<T>;

  namespace views { inline constexpr unspecified unchecked_take = unspecified; }    // freestanding
  […]
  namespace views { inline constexpr unspecified drop = unspecified; }              // freestanding

  // [range.unchecked.drop], unchecked_drop_view
  template<view V> class unchecked_drop_view;                                       // freestanding

  template<class T>
    constexpr bool enable_borrowed_range<unchecked_drop_view<T>> =                  // freestanding
      enable_borrowed_range<T>;

  namespace views { inline constexpr unspecified unchecked_drop = unspecified; }    // freestanding
  […]
}
```

3. Add **25.7.? Unchecked take view [range.unchecked.take]** after 25.7.10 [range.take] as indicated:

[25.7.?.1] Overview [range.unchecked.take.overview]

-1- `unchecked_take_view` produces a view of the first *N* elements from another view. The behavior is undefined if the adapted view contains fewer than *N* elements.

-2- The name `views::unchecked_take` denotes a range adaptor object (25.7.2 [range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::unchecked_take(E, F)` is ill-formed. Otherwise, the expression `views::unchecked_take(E, F)` is expression-equivalent to:

(2.1) — *Preconditions*: `static_cast<D>(F) >= 0` is `true` and `E` has at least `static_cast<D>(F)` elements.

> [*Drafting note:* The *Preconditions* is to prevent use cases like `views::iota(0, 5) | views::unchecked_take(10)` from being well-defined.]

(2.2) — If `T` models `random_access_range` and is a specialization of `span` (23.7.2.2 [views.span]), `basic_string_view` (27.3 [string.view]), or `ranges::subrange` (25.5.4 [range.subrange]), then `U(ranges::begin(E), ranges::begin(E) + static_cast<D>(F))`, except that `E` is evaluated only once, where `U` is a type determined as follows:

(2.2.1) — if `T` is a specialization of `span`, then `U` is `span<typename T::element_type>`;

(2.2.2) — otherwise, if `T` is a specialization of `basic_string_view`, then `U` is `T`;

(2.2.3) — otherwise, `T` is a specialization of `subrange`, and `U` is `subrange<iterator_t<T>>`;

(2.3) — otherwise, if `T` is a specialization of `iota_view` (25.6.4.2 [range.iota.view]) that models `random_access_range`, then `iota_view(*ranges::begin(E), *(ranges::begin(E) + static_cast<D>(F)))`, except that `E` is evaluated only once.

(2.4) — Otherwise, if `T` is a specialization of `repeat_view` (25.6.5.2 [range.repeat.view]), then `views::repeat(*E.value_, static_cast<D>(F))`.

(2.5) — Otherwise, `unchecked_take_view(E, F)`.

-3- [*Example 1*:

```
auto ints = views::iota(0) | views::unchecked_take(10);
for (auto i : ints | views::reverse) {
  cout << i << ' ';  // prints 9 8 7 6 5 4 3 2 1 0
}
```

— *end example]*

[25.7.?.2] Class template `unchecked_take_view` [range.unchecked.take.view]

```
namespace std::ranges {
  template<view V>
  class unchecked_take_view : public view_interface<unchecked_take_view<V>> {
  private:
    V base_ = V();                                          // exposition only
    range_difference_t<V> count_ = 0;                       // exposition only

  public:
    unchecked_take_view() requires default_initializable<V> = default;
    constexpr explicit unchecked_take_view(V base, range_difference_t<V> count);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin() requires (!simple-view<V>) {
      if constexpr (random_access_range<V>)
        return ranges::begin(base_);
      else
        return counted_iterator(ranges::begin(base_), count_);
    }

    constexpr auto begin() const requires range<const V> {
      if constexpr (random_access_range<const V>)
        return ranges::begin(base_);
      else
        return counted_iterator(ranges::begin(base_), count_);
    }

    constexpr auto end() requires (!simple-view<V>) {
      if constexpr (random_access_range<V>)
        return ranges::begin(base_) + count_;
      else
        return default_sentinel;
    }

    constexpr auto end() const requires range<const V> {
      if constexpr (random_access_range<const V>)
        return ranges::begin(base_) + count_;
      else
        return default_sentinel;
    }

    constexpr auto size() const noexcept { return to-unsigned-like(count_); }
  };

  template<class R>
    unchecked_take_view(R&&, range_difference_t<R>)
      -> unchecked_take_view<views::all_t<R>>;
}

constexpr explicit unchecked_take_view(V base, range_difference_t<V> count);
```

-1- *Preconditions*: `count >= 0` is `true` and `base` has at least `count` elements.

-2- *Effects*: Initializes *`base_`* with `std::move(base)` and *`count_`* with `count`.

4. Add **25.7.? Unchecked drop view [range.unchecked.drop]** after 25.7.12 [range.drop] as indicated:

[25.7.?.1] Overview [range.unchecked.drop.overview]

-1- `unchecked_drop_view` produces a view excluding the first *`N`* elements from another view. The behavior is undefined if the adapted view contains fewer than *`N`* elements.

-2- The name `views::unchecked_drop` denotes a range adaptor object (25.7.2 [range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::unchecked_drop(E, F)` is ill-formed. Otherwise, the expression `views::unchecked_drop(E, F)` is expression-equivalent to:

(2.1) — *Preconditions*: `static_cast<D>(F) >= 0` is `true` and `E` has at least `static_cast<D>(F)` elements.

(2.2) — If `T` models `random_access_range` and is

(2.2.1) — a specialization of `span` (23.7.2.2 [views.span]),

(2.2.2) — a specialization of `basic_string_view` (27.3 [string.view]),

(2.2.3) — a specialization of `iota_view` (25.6.4.2 [range.iota.view]), or

(2.2.4) — a specialization of `subrange` (25.5.4 [range.subrange]) where `T::`*`StoreSize`* is `false`,

then `U(ranges::begin(E) + static_cast<D>(F), ranges::end(E))`, except that `E` is evaluated only once, where `U` is `span<typename T::element_type>` if `T` is a specialization of `span` and `T` otherwise.

(2.3) — Otherwise, if `T` is a specialization of `subrange` (25.5.4 [range.subrange]) that models `random_access_range` and `sized_range`, then `T(ranges::begin(E) + static_cast<D>(F), ranges::end(E), `*`to-unsigned-like`*`(ranges::distance(E) - static_cast<D>(F)))`, except that `E` and `F` are each evaluated only once.

(2.4) — Otherwise, if `T` is a specialization of `repeat_view` (25.6.5.2 [range.repeat.view]):

(2.4.1) — if `T` models `sized_range`, then

```
views::repeat(*E.value_, ranges::distance(E) - static_cast<D>(F))
```

except that `E` is evaluated only once;

(2.4.2) — otherwise, `((void)F, `*`decay-copy`*`(E))`, except that the evaluations of `E` and `F` are indeterminately sequenced.

(2.5) — Otherwise, `unchecked_drop_view(E, F)`.

-3- [*Example 1*:

```
vector<int> v{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
for (int i : is | views::unchecked_drop(6) | views::unchecked_take(3)) {
  cout << i << ' '; // prints 6 7 8
}
```

— *end example]*

[25.7.?.2] Class template `unchecked_drop_view` [range.unchecked.drop.view]

```
namespace std::ranges {
  template<view V>
  class unchecked_drop_view : public view_interface<unchecked_drop_view<V>> {
  public:
    unchecked_drop_view() requires default_initializable<V> = default;
    constexpr explicit unchecked_drop_view(V base, range_difference_t<V> count);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin()
      requires (!(simple-view<V> && random_access_range<const V>));
    constexpr auto begin() const requires random_access_range<const V>;

    constexpr auto end() requires (!simple-view<V>)
    { return ranges::end(base_); }

    constexpr auto end() const requires range<const V>
    { return ranges::end(base_); }

    constexpr auto size() requires sized_range<V>
    { return ranges::size(base_) - static_cast<range_size_t<V>>(count_); }

    constexpr auto size() const requires sized_range<const V>
    { return ranges::size(base_) - static_cast<range_size_t<const V>>(count_); }
```

```
  private:
    V base_ = V();                                      // exposition only
    range_difference_t<V> count_ = 0;                   // exposition only
  };

  template<class R>
    unchecked_drop_view(R&&, range_difference_t<R>)
      -> unchecked_drop_view<views::all_t<R>>;
}
```

```
constexpr explicit unchecked_drop_view(V base, range_difference_t<V> count);
```

-1- *Preconditions*: `count >= 0` is `true` and `base` has at least `count` elements.

-2- *Effects*: Initializes *`base_`* with `std::move(base)` and *`count_`* with `count`.

```
constexpr auto begin() requires (!(simple-view<V> && random_access_range<const V>));
constexpr auto begin() const requires random_access_range<const V>
```

-3- *Returns*: `ranges::next(ranges::begin(base_), count_)`.

-4- *Remarks*: In order to provide the amortized constant-time complexity required by the `range` concept when `unchecked_drop_view` models `forward_range`, the first overload caches the result within the `unchecked_drop_view` for use on subsequent calls.

[*Note 1*: Without this, applying a `reverse_view` over a `unchecked_drop_view` would have quadratic iteration complexity. — *end note*]

References

[P2760R1]
Barry Revzin. A Plan for C++26 Ranges. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r1.html